

ABYSS Project Briefing Simple A to Z

AI powered side scan sonar detection, backend, frontend, and pipeline explanation

Prepared from the current repository on 12 September 2026

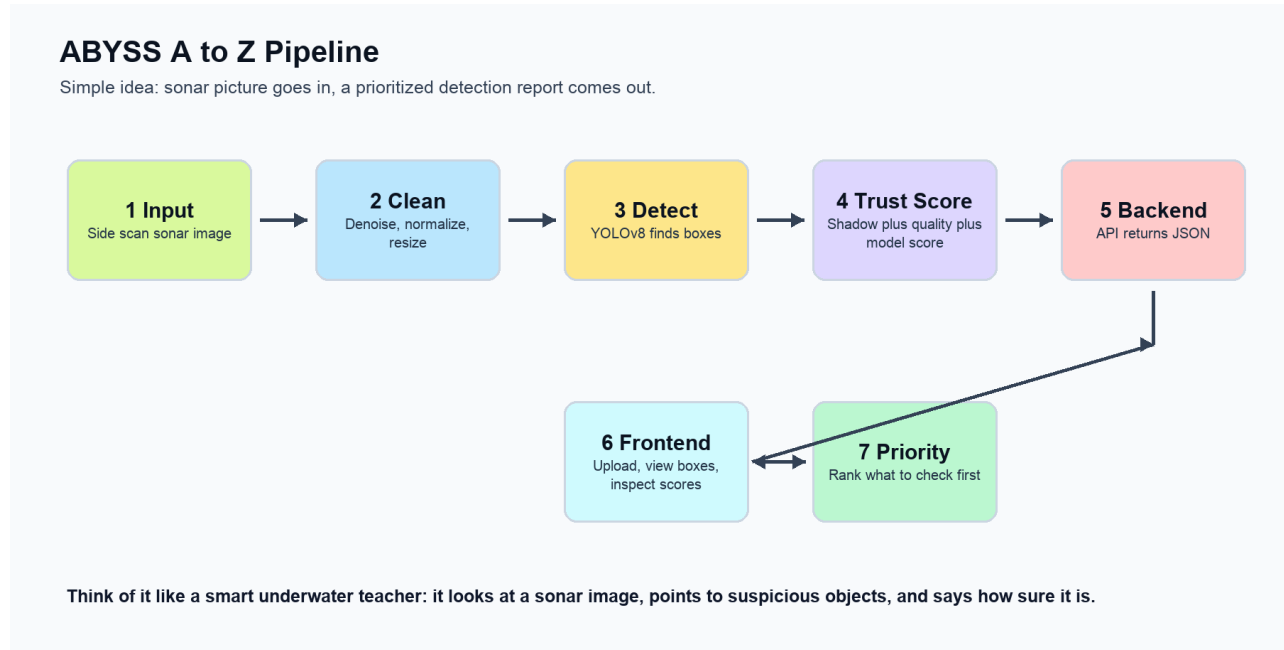


Figure 1. The whole ABYSS idea in one picture.

This briefing explains the project in very simple language so it can be presented confidently. Imagine ABYSS as a smart underwater helper. You give it a sonar picture. It cleans the picture, looks for objects like planes, ships, and shipwrecks, gives each finding a confidence score, and shows the results in a dashboard.

The current honest status is: Phase 0 to Phase 4 have been audited for the ML and backend work. A React frontend prototype exists and works with the backend, but the project notes say the final Phase 5 dashboard is not formally audited yet.

One Minute Presentation Story

ABYSS helps people inspect underwater sonar images faster. Sonar images are like blurry underwater shadows. A human can inspect them, but it takes time. ABYSS uses an AI detector and extra trust checks to point out suspicious objects and rank what should be checked first.

- Input: a side scan sonar image.
- Processing: clean and resize the image to 640 by 640 pixels.
- Detection: YOLOv8 draws boxes around possible objects.
- Confidence: the backend combines model confidence, image quality, and sonar shadow clues.
- Output: the dashboard shows boxes, scores, coordinates, and priority ranking.
- Human role: the system helps the operator decide what to inspect first; it does not replace expert review.

A to Z What We Have Done

Letter	Topic	Simple explanation
A	Aim	Build a sonar object detection system for marine debris and underwater anomalies.
B	Base project	Set up Python ML code, FastAPI backend, React frontend, tests, requirements, and project docs.
C	Collected data structure	Prepared raw, split, processed, experiment, and log folders for sonar datasets.
D	Data ingestion	Scripts copy datasets, read labels, count classes, and make manifests.
E	Data splitting	Images are split into train, validation, and test by inferred group so the same group does not appear in two splits.
F	Preprocessing	Images are denoised, normalized, and letterboxed to 640 pixels for YOLO.
G	Detector	YOLOv8 detects Plane, Ship, and Shipwreck classes.
H	Better checkpoint	The current deployed checkpoint is improved_v2_baseline_finetune_adamw.pt.
I	Shadow features	The system checks whether a detected object has a reasonable dark sonar shadow nearby.
J	Image quality	The system calculates if the image area is clear enough to trust.
K	Calibration	A calibration file turns raw scores into a more interpretable final probability.
L	Evaluation	We measured held out test metrics, FROC, ECE calibration, precision, recall, F1, and confusion.
M	Experiments	We tried bounded retrains for shipwreck loss, shipwreck

Letter	Topic	Simple explanation
		augmentation, box weight, and mosaic schedule.
N	Decision	We kept the baseline 640 v4 model as deployed because experiments had tradeoffs.
O	Backend	FastAPI serves health, detect, geolocate, and priority endpoints.
P	Frontend	React dashboard uploads sonar images, shows boxes, displays score bars, geolocates detections, and ranks priorities.
Q	Quality honesty	We removed a filename based Plane override so the backend reports native model output honestly.

Pipeline Explained Like You Are 10

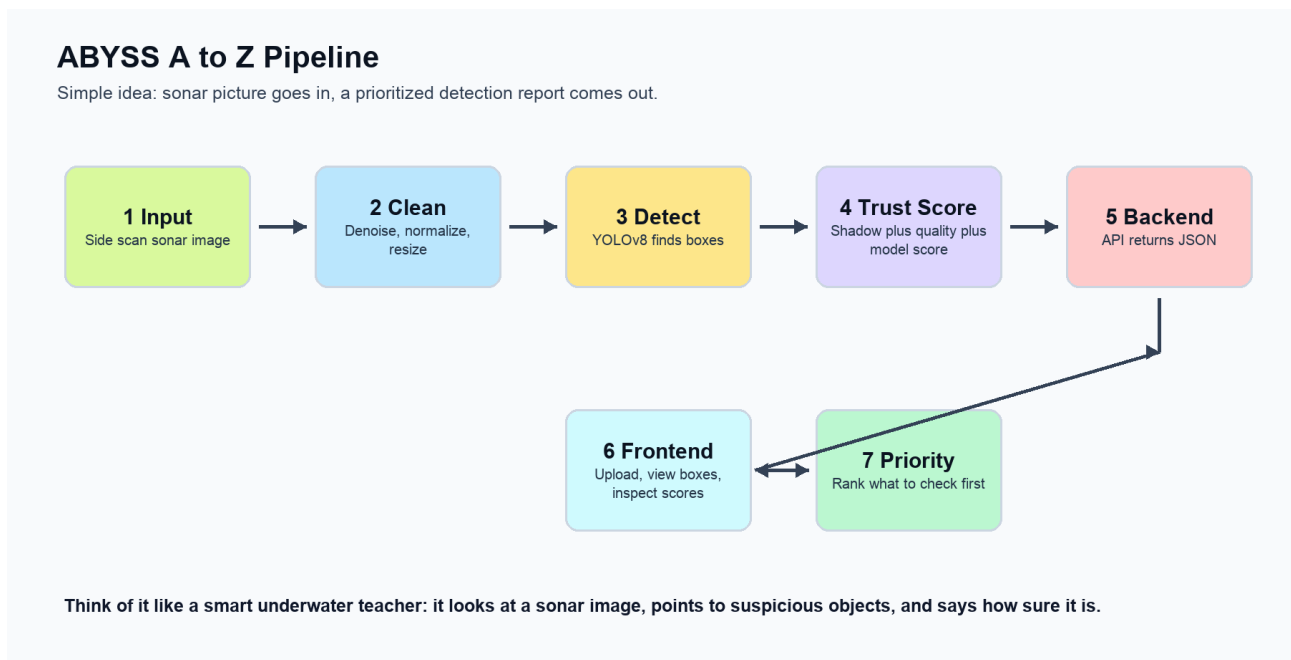


Figure 2. The ABYSS pipeline from input image to operator decision.

Think of the pipeline like washing, reading, checking, and reporting a worksheet.

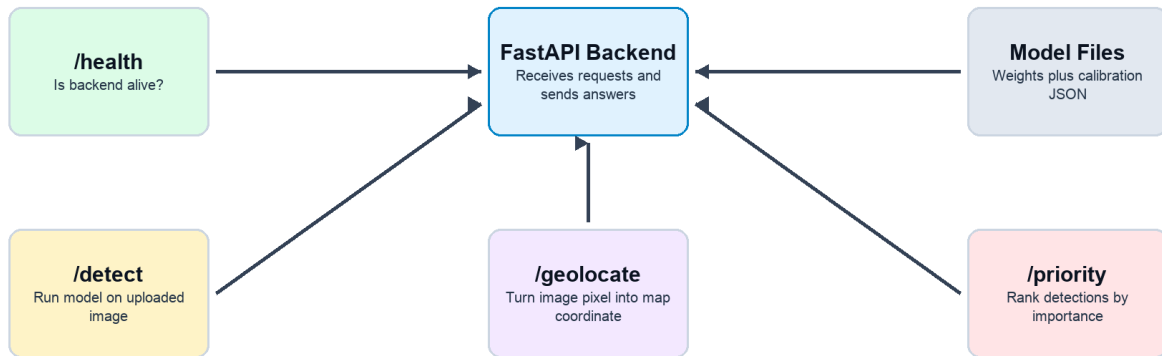
- First, ABYSS gets a sonar image. A sonar image is made from sound, not normal light.
- Second, it cleans the image because sonar pictures can be noisy.
- Third, YOLOv8 looks at the image and draws boxes around likely objects.
- Fourth, ABYSS asks extra questions: Is the picture clear? Does the object have a shadow that makes sense for sonar?
- Fifth, the backend sends a neat JSON answer to the frontend.

- Sixth, the frontend shows the result in a way a human operator can understand.

Backend Complete Idea

Backend Framework

FastAPI is the waiter between the frontend and the AI model.



Important honesty: geolocation is real only when real navigation metadata is supplied. Without it, the backend clearly labels the coordinate as simulated.

Figure 3. Backend endpoints and how they fit together.

The backend is the brain room. It is written with FastAPI. The frontend asks it questions, and the backend answers with JSON data.

Endpoint	What it does	Simple example
/health	Checks whether the backend is alive.	The frontend shows Backend reachable or Backend unavailable.
/detect	Receives an uploaded image and runs YOLOv8 plus confidence scoring.	Returns boxes, class names, confidence scores, and latency.
/geolocate	Converts the middle of a detection box into latitude and longitude.	Real if navigation metadata is supplied; simulated fallback otherwise.
/priority	Sorts detections by a transparent formula.	Uses confidence, hazard class weight, object size, and optional distance to sensitive zone.

The backend loads two important files: the trained model weights and the calibration JSON. It checks that the calibration file matches the model, image size, preprocessing mode, and model hash. This matters because using the wrong calibration with the wrong model would make the confidence numbers untrustworthy.

Detection flow inside the backend:

- Read the uploaded image bytes and decode them with OpenCV.

- Clean and resize the image using the same baseline preprocessing used during evaluation.
- Run YOLOv8 on the 640 pixel image.
- For each box, calculate shadow features and image quality.
- Build a three number feature vector: detector confidence, shadow consistency, image quality.
- Use logistic fusion and temperature calibration to produce composite confidence.
- Return a list of detections with class name, box coordinates, raw confidence, shadow score, quality score, fused probability, calibrated probability, and composite confidence.

Backend score	Meaning in simple language
detector_confidence	How sure YOLO is that the object exists.
image_quality_score	How clear and useful the image area looks.
shadow_consistency_score	Whether the dark sonar shadow looks believable.
logistic_fused_probability	A learned combination of the three clues.
temperature_calibrated_probability	The final adjusted probability used as composite confidence.
priority_score	Operational ranking score, not the same as detection probability.

Frontend Complete Idea

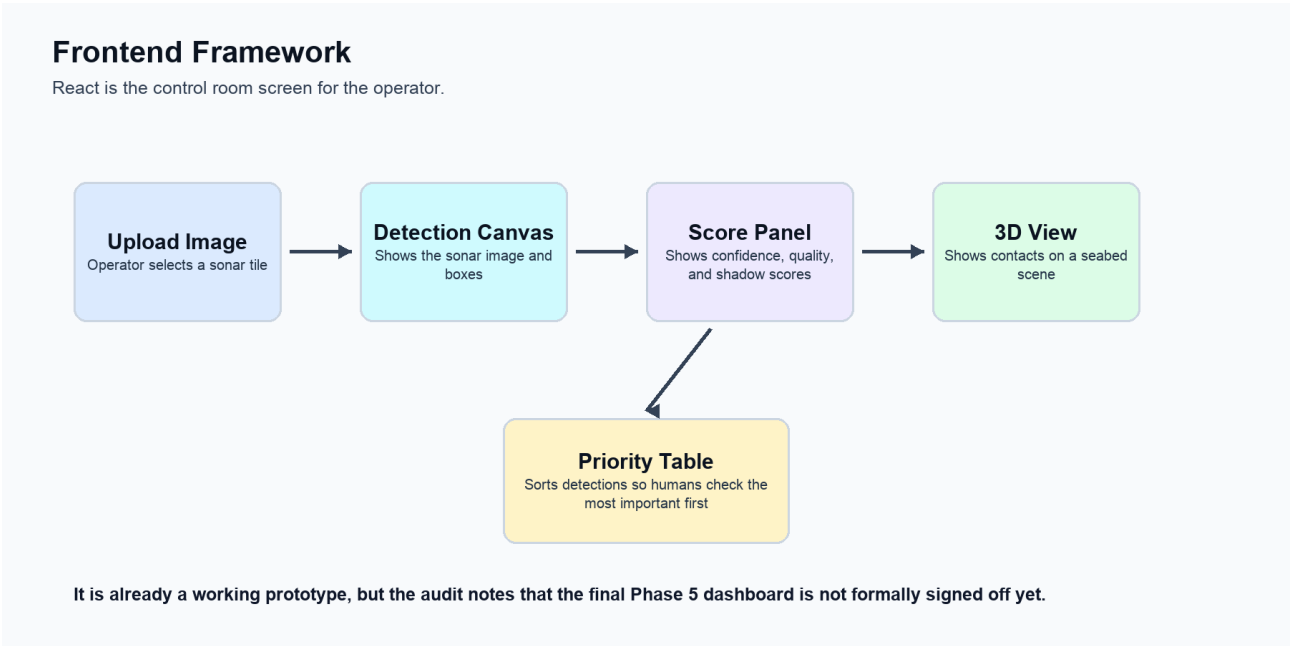


Figure 4. Frontend operator workflow.

The frontend is the control room screen. It is built with React, TypeScript, Vite, Tailwind CSS, lucide icons, and React Three Fiber for the 3D scene.

Frontend part	What the user sees	What it talks to
Upload form	Choose a sonar image and press Detect.	/detect

Frontend part	What the user sees	What it talks to
Sonar canvas	Image with detection boxes and labels drawn on top.	Uses /detect response
Confidence breakdown	Bar charts for detector, image quality, shadow, and composite confidence.	Uses /detect response
3D contact view	Simple seabed scene with markers for detections.	/geolocate
Priority table	Ranked list of what to inspect first.	/priority
Backend status	Reachable or unavailable indicator.	/health

In presentation language: the frontend does not do the AI thinking itself. It sends the image to the backend, receives the answer, and makes the answer easy for a human to inspect.

Machine Learning Framework

The ML framework is centered on YOLOv8. YOLO means You Only Look Once. In simple words, it looks at the image once and predicts boxes and classes quickly.

Layer	Tool or file	Simple purpose
Data	abyss/data and data.yaml	Stores train, validation, and test images and labels.
Preprocessing	preprocess.py	Makes images cleaner and the same size.
Training	train_yolo.py and train_bounded_candidate.py	Creates or experiments with model checkpoints.
Inference	infer.py and backend services.py	Runs the model on new images.
Calibration	fit_calibration.py and calibration_improved_v4.json	Makes confidence scores easier to trust.
Evaluation	evaluate.py, evaluate_froc.py, evaluate_calibration.py	Measures accuracy, false alarms, and calibration.

The classes are Plane, Ship, and Shipwreck. The model is best at Ship, decent at Plane by AP, and currently weak on Shipwreck. That weakness should be mentioned honestly.

What The Results Say

Class	Ground truth objects	AP at IoU 0.5	AP 0.5 to 0.95	Simple meaning
Plane	17	0.779424	0.378747	Often finds planes, but many are confused with ships at the deployed threshold.
Ship	74	0.957086	0.674665	Strongest class.
Shipwreck	20	0.144992	0.058058	Weak class and needs more work.
Overall	111	0.627168	0.370490	Useful prototype, not perfect production

Class	Ground truth objects	AP at IoU 0.5	AP 0.5 to 0.95	Simple meaning
				truth.

At the deployed raw confidence threshold of 0.25, the held out test results were 77 true positives, 44 false positives, and 34 false negatives. Precision was 0.636364, recall was 0.693694, and F1 was 0.663793.

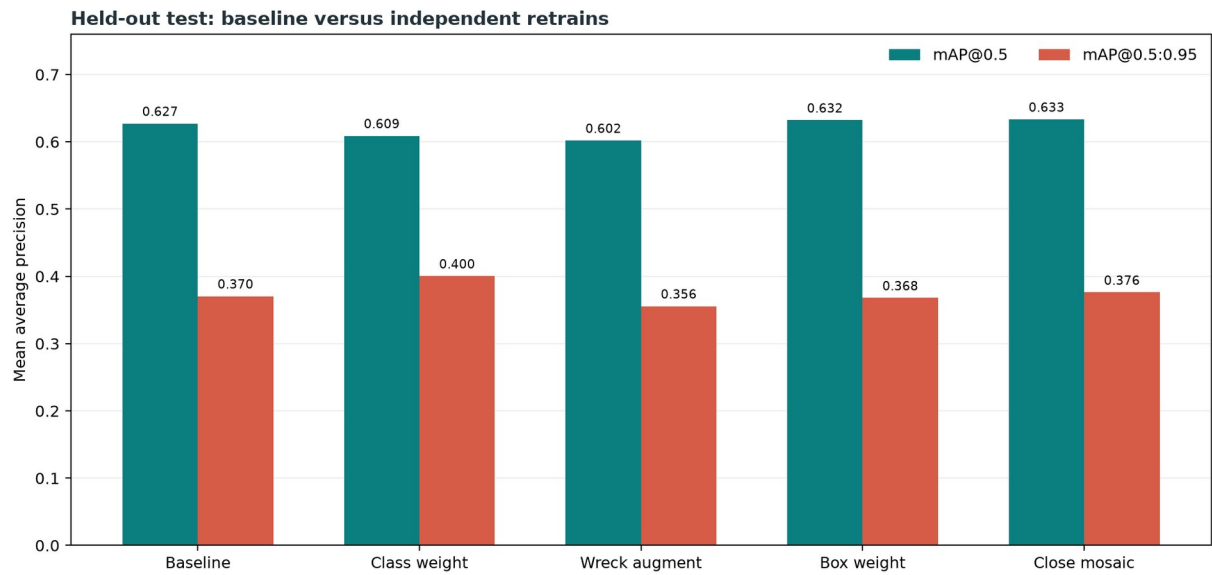


Figure 5. Overall held out mAP comparison for baseline and bounded experiments.

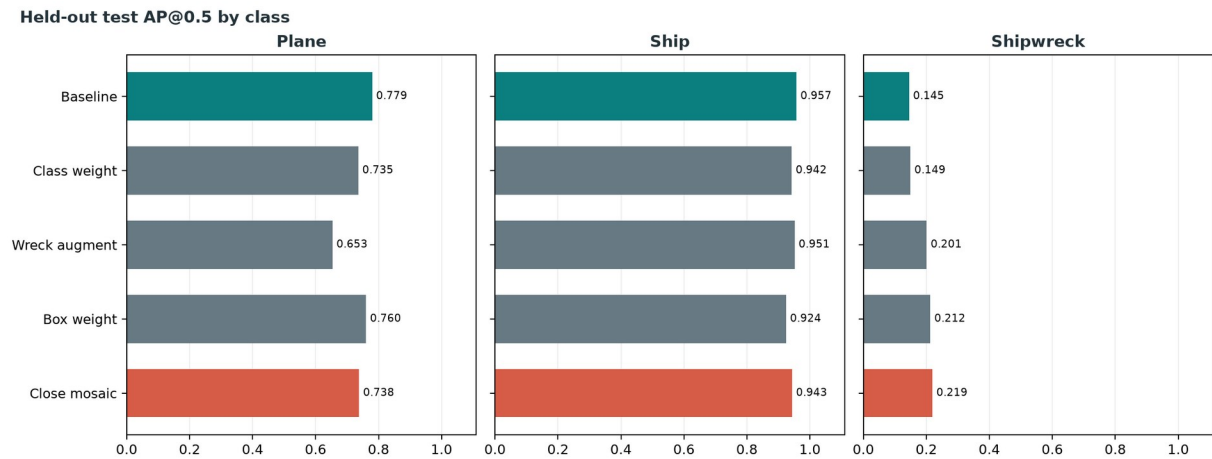


Figure 6. Per class held out AP at IoU 0.5.

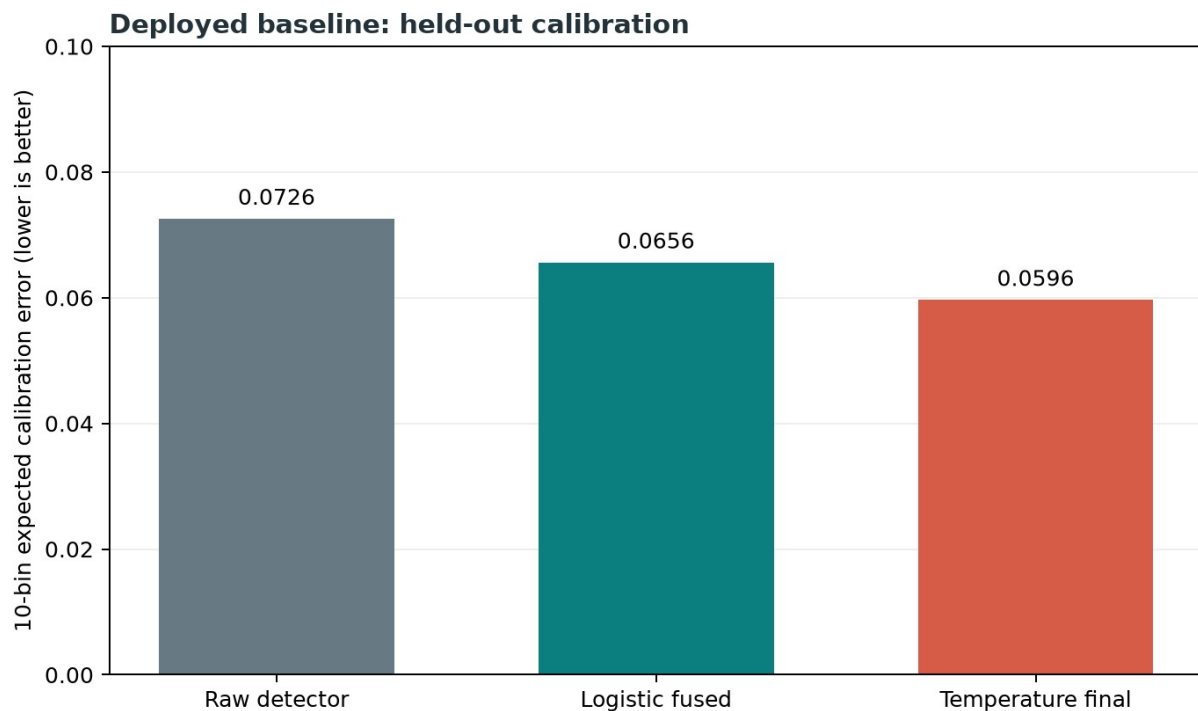


Figure 7. Calibration comparison. Lower ECE means confidence is better aligned with correctness.

Bounded retrain decision flow

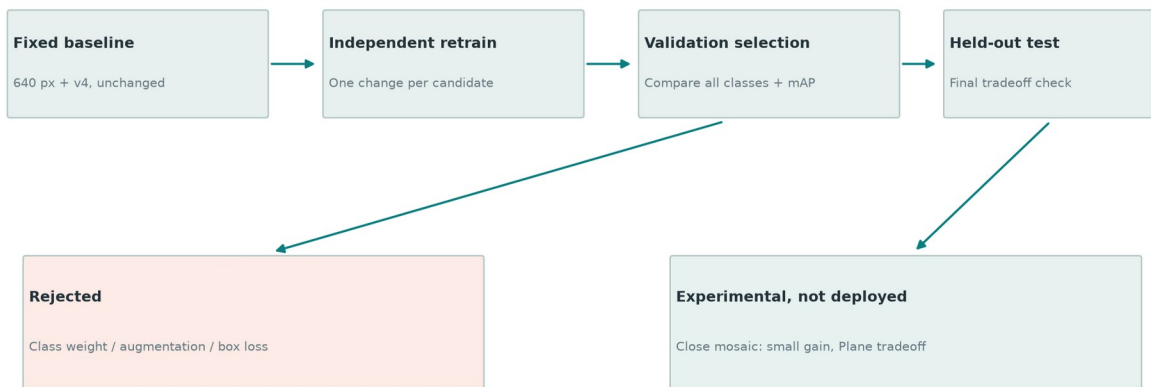


Figure 8. Why the deployed model was kept instead of switching to an experimental candidate.

Experiment Story

After the baseline, we tried careful bounded experiments. Bounded means we changed one idea at a time so we could understand what helped or hurt.

Candidate	Goal	Outcome
Shipwreck loss weight	Make the model care more about shipwrecks.	Shipwreck improved slightly, but overall validation and some class tradeoffs were not good enough.
Shipwreck only augmentation	Create more shipwreck examples.	Shipwreck improved, but Plane and overall results dropped.
Box weight 9.0	Encourage better box placement.	Some AP improved, but main target mAP 0.5 to 0.95 dropped.
Close mosaic 15	Change mosaic augmentation schedule.	Small aggregate gain, but Plane and Ship AP 0.5 fell, so kept as experimental only.

Final decision: keep baseline 640 v4 deployed. Any future model change must get a new calibration file and a full rerun of corrected metrics.

Important Limitations To Say Clearly

- Shipwreck detection is still weak and needs more data, better training, or a better strategy.
- The model can confuse Plane and Ship. Earlier code had a filename based Plane override, but it has been removed so results are honest.
- Geolocation is real only when survey navigation metadata is provided. Without it, the backend uses a simulated fallback origin and labels it as simulated.
- We cannot calculate FP per kilometer, FP per square kilometer, or positional RMSE because real survey distance, area, sensor specs, and GPS ground truth are missing.
- Priority score is a transparent policy formula, not a learned danger probability.
- The React dashboard works as a prototype, but the audit says final Phase 5 dashboard work has not been formally signed off.

How To Explain Frameworks In The Presentation

Framework	Child friendly explanation	Why ABYSS uses it
FastAPI	A fast waiter that takes requests and brings answers.	Serves detection, geolocation, priority, and health APIs.
React	A way to build a screen from small reusable blocks.	Builds the operator dashboard.
Vite	A quick starter engine for the web app.	Runs the frontend during development.
Tailwind CSS	Small ready made style rules.	Makes the dashboard clean without writing huge CSS files.
React Three Fiber	React for 3D scenes.	Shows the seabed contact view.
OpenCV	A computer vision toolbox.	Reads, cleans, resizes, and measures sonar images.
YOLOv8	An AI model that draws boxes around objects.	Detects Plane, Ship, and Shipwreck.
Pydantic	A strict form checker.	Validates API request and response shapes.

Suggested Spoken Script

Here is a simple way to present the project:

Our project is called ABYSS. It helps inspect side scan sonar images.

Side scan sonar images are not normal photos. They are made from sound, so they can be noisy and hard to read.

First we clean the image and resize it to 640 by 640 pixels.

Then YOLOv8 looks for three object types: Plane, Ship, and Shipwreck.

After that, we do not blindly trust the model. We also check image quality and sonar shadow clues.

The backend combines those clues into a final confidence score and sends the result as JSON.

The frontend lets a user upload an image, see boxes on the sonar image, inspect confidence bars, see a 3D contact view, and read a priority list.

Our current model is strongest on Ship, okay on Plane by AP, and still weak on Shipwreck. We are honest about that because this is an engineering project, not magic.

The next best work is improving shipwreck data and retraining with proper calibration.

Quick Q and A

Question	Good answer
What problem are we solving?	We help humans quickly find suspicious underwater objects in sonar images.
Is it fully automatic?	No. It is a decision support tool. Humans still verify important detections.
What does backend do?	It runs detection, scoring, geolocation, and priority ranking.
What does frontend do?	It makes backend answers visible and understandable.
What is the pipeline?	Input sonar image, clean image, detect objects, calculate trust scores, serve API result, show dashboard.
What is the biggest weakness?	Shipwreck detection and lack of real navigation metadata.
What is the biggest strength?	The project is end to end: data pipeline, detector, calibration, backend API, and frontend prototype.

Project File Map

Folder or file	What it contains
README.md	High level architecture, roadmap, quickstart, and API overview.
abyss/ml/preprocessing	Data ingestion, splitting, denoising, normalizing, resizing, label conversion.
abyss/ml/detection	YOLO training, inference, evaluation, bounded experiments, model weights.
abyss/ml/shadow_confidence	Shadow features, image quality, calibration, ECE, FROC, threshold checks.

Folder or file	What it contains
abyss/backend/app	FastAPI routes, schemas, and backend service logic.
frontend/src/App.tsx	Main React dashboard prototype.
docs/METRICS.md	Canonical current deployed metrics.
docs/PROJECT_METRICS.md	Combined work summary, charts, and experimental comparison.
logs	Saved experiment reports and audit outputs.

Final Summary

ABYSS is an end to end sonar detection project. It is not just a model file. It includes data preparation, model training, confidence calibration, backend APIs, evaluation reports, experiment audits, and a frontend prototype. The best way to present it is as a human assist tool: it helps an operator find and prioritize suspicious underwater objects faster.

The correct final message is confident but honest: we built a working pipeline and prototype, measured it carefully, kept the safest deployed model, and documented what still needs work.